AdaMacro: Budgeted Skill Discovery by Consolidating Frequent Tool Trajectories

1、 Introduction

大语言模型驱动的工具调用型 Agent 已成为自动化复杂任务的重要方式。尽管任务指令千差万别，实际交互轨迹显示，Agent 经常重复执行相似的短程子流程，例如“检索后选择目标并打开”、“抓取后解析并写入”或“读取文件后运行命令”。这些重复片段表明，若能将高频短程步骤固化为可复用单元，便有机会缩短有效决策链路，降低探索成本，并显著提升训练时的样本利用率。

近期经验重用方向主要依赖 Memory 与自我进化框架。常见做法是将历史交互片段检索作为上下文参考，或将经验总结为文字建议注入提示，从而辅助模型规划。与此同时，FaSTA* 等工作开始探索将复用单元进一步推进到动作层，尝试挖掘可执行的子程序。这一趋势表明，从“文本记忆的上下文层复用”走向“结构化的动作层复用”是一条提升 Agent 效率的自然路径。

然而，将复用推进到动作层并不等价于把经常相邻的工具名直接拼接成新动作。**第一**，基于局部共现的朴素拼接会导致动作空间膨胀。若缺乏明确的预算与筛选机制，近似变体会导致宏库规模迅速增长，反而增加了选择难度与冲突概率。**第二**，纯符号拼接难以提供一致的执行语义。宏动作不仅是顺序列表，还需具备“候选选择”、“异常处理”与“中断回退”能力；否则，一旦宏在高频使用中失败，不仅难以定位具体错误阶段，强化学习的训练信号也难以精确分配到宏内部。因此，核心难点在于如何在基数约束下，选出最具压缩收益的片段，并将其转化为既可执行又可诊断的技能。

为此，我们提出 **AdaMacro**。我们将宏发现建模为一个带基数约束的行为压缩问题：在给定预算下，从成功轨迹中利用 BPE 式迭代合并算法，优先挖掘高频且高收益的子流程，并将其转化为模板化的可执行技能（Executable Skills）注册到工具库中。为了解决执行与训练难题，AdaMacro 采用两项关键设计：其一，保留原子工具与宏技能并存，使 Agent 可灵活决策；其二，在运行时注入 Trace 追踪与软中断机制，使宏具备可定位、可回退的统一语义，并支持基于 SFT 与 GRPO 的分阶段反馈训练。实验表明，该方法在有效控制动作空间规模的同时，显著提升了 Agent 的执行效率与鲁棒性。

第一，我们提出带约束的 BPE 式宏挖掘，在成功轨迹上自动发现高频子流程，并用词表预算与轻量裁剪控制宏库规模。

第二，我们提出模板化的技能（skills）实例化与 trace 感知执行机制，将工具宏转化为语义明确的可执行技能，统一候选选择，异常处理，中断与回退，并与原子工具完全并存兼容。

第三，我们在 SFT + GRPO 框架下引入“宏整体奖励 + 宏内部阶段反馈”，利用组内相对比较稳定学习混合动作空间中的宏选择与内部策略参数选择，并在 TOOLATHLON 与 TOUCAN 上系统评估效率与性能收益。

2、 Method

2.1 带预算约束的 BPE 宏挖掘

我们从训练日志中抽取成功轨迹，将每条轨迹映射为工具 token 序列。为减少具体参数值噪声并提升泛化能力，每个原子工具调用被抽象为“工具名 + 关键参数签名”的 token，例如 Search[q]、Click[id]。在该表示下，宏挖掘被建模为一个带预算的序列压缩过程。我们采用 BPE 式迭代合并：统计语料中相邻 token 对的出现次数，并在每轮合并中选择计数最大的相邻对进行合并。

给定轨迹语料$\mathcal{D}$，相邻对(u, v)的加权计数定义为：

$$C(u,v) = \sum_{\tau \in D} \sum_{t=1}^{|\tau|-1} 1[x_t = u, x_{t+1} = v],$$

其中 $w(\tau)$默认对成功轨迹取 1。每轮选择出现最频繁的相邻对并合并为新 token：

$$(u^*, v^*) = \underset{(u,v)}{\operatorname{argmax}} C(u,v).$$

在理论层面，BPE 可以视作“在给定合并轮数预算下最大化压缩收益”的贪心算法。相关论文的分析表明，BPE 背后的最优问题具有近似难度，但贪心 BPE 依然能对压缩收益提供常数因子近似保证。 在我们的场景中，工具序列同样是离散符号串，因此这一保证可作为选择 BPE 的理论支撑。我们需要补充一个对照实验，将 BPE 与“频繁 n-gram 挖掘并选 top-K 宏”的方法直接比较，突出 BPE 在预算下得到更紧凑、冗余更低的宏库这一点。

2.2 自动技能实例化与 trace 感知

BPE 产出的是符号层面的宏 token。为使 LLM 能像选择普通工具一样选择宏，我们将每个宏 token m实例化为可执行技能（skills） S_m，并注册到工具库中，同时保留全部原子工具。技能实例化采用模板化方式实现。模板集合覆盖常见结构，例如纯顺序执行模板与“上一步输出候选列表，下一步需从候选中选取输入”的选择模板。该设计避免直接任意代码生成带来的不确定性，并保证技能执行语义一致。

技能对外暴露的参数自动确定：基于工具 schema 与相邻步骤字段匹配，内部可由前序输出满足的字段被隐藏，仍需策略提供的输入字段被提升为技能形式参数。为保留有限的内部可控性，我们额外暴露离散化 select_strategy 参数，用于控制候选选择方式（例如 rank-0、rank-1、random、filter）。也就是说为了不让宏变得太死板，我们在自动传递数据的同时，给 LLM 留了一个外部参数，让它能指挥宏在内部怎么挑数据。技能内部的中间变量由解释器自动传递，不要求模型显式填写。

运行时，我们对技能执行注入 trace。技能解释器按序展开内部原子工具调用，并记录每步工具名与状态码：

$$\mathrm{Trace}(S_m) = [(\mathrm{tool}_1, \mathrm{status}_1), \dots, (\mathrm{tool}_k, \mathrm{status}_k)].$$

若执行出现硬失败（超时、报错）或逻辑阻断（候选为空、断言失败），解释器触发软中断，停止后续展开并返回包含 trace 与中间观测的中断结果，将控制权交还给策略。由于原子工具始终保留，策略可退回原子动作逐步完成任务，或改选其他技能继续推进。

2.3 SFT + GRPO 训练与分阶段奖励

在完成技能实例化后，训练的关键变成如何在“技能与原子并存”的动作集合里稳定学会选择。AdaMacro 采用两阶段训练。首先用 SFT 学习基本工具调用格式，并在工具列表中显式提供技能，使模型从一开始就把技能当作正常候选。随后进入强化学习阶段，采用 GRPO 做策略优化。AdaMacro 在混合动作空间中训练策略，动作集合由原子工具与技能并存构成：

$$\mathcal{A} = \mathcal{A}_{\mathrm{atom}} \cup \mathcal{A}_{\mathrm{skill}}.$$

训练采用两阶段流程。SFT 阶段学习基础工具调用格式与常见模式，并让模型从初始化阶段理解技能也是可选工具。随后 RL 阶段使用 GRPO 优化策略，学习在不同上下文下选择技能或原子工具，并学习技能内部离散策略参数的选择。

选择 GRPO 的关键原因在于混合动作空间会显著增加“同一状态下的可行轨迹数量”。面对同一提示与相同上下文，通常同时存在用技能快速推进的有效轨迹与用原子工具逐步完成的有效轨迹。此时最直接、最稳定的学习信号来自这些可行轨迹之间的相对优劣比较。GRPO 对同一提示进行组采样，并在组内构造相对优势来更新策略，从而更适合该决策环境。具体目标为

$$\max_{\theta} \mathbb{E}_p[\frac{1}{G}\sum_{i=1}^{G} A_i \log \pi_\theta(y_i \mid p) - \beta \mathrm{KL}(\pi_\theta(\cdot \mid p) \| \pi_{\mathrm{ref}}(\cdot \mid p))], A_i = \mathrm{normalize}(R_i).$$

回报由任务级成功与技能级反馈共同构成。技能完整成功时给整体奖励，技能中断时依据 trace 的中断位置给出阶段性反馈，避免把所有失败都压缩成一个不可解释的负信号：

$$R(y) = R_{\mathrm{task}}(y) + \lambda R_{\mathrm{skill}}(y), R_{\mathrm{skill}}(y) = \begin{cases} r_{\mathrm{complete}}, & \text{技能完整成功} \\ (j-1)\, r_{\mathrm{pass}} + r_{\mathrm{fail}}^{(j)}, & \text{在第 } j \text{ 步中断.} \end{cases}$$

该设计使训练信号携带失败位置与失败类型信息，并结合软中断机制保证策略可在技能失败后退回原子工具继续探索。

3 Experiment

3.1 数据集与设置

我们在 TOOLATHLON 与 TOUCAN 上进行评估。宏挖掘仅使用训练集成功轨迹构建技能库，评估阶段冻结技能库以保证可复现。为保证公平比较，我们统一 backbone、工具接口与训练预算，并以真实原子工具调用次数作为主要预算约束标准，避免技能降低决策次数带来的不公平优势。预计在 Qwen2.5-7b、Qwen3-4b、Llama3.2-3b 上进行实验。

3.2 对比方法

我们按分类的形式组织基线：仅原子工具的 RL/SFT-RL 方法；基于检索的 raw-context memory；基于反思的显式指导 memory；自我进化式记忆维护方法；以及具备可执行技能或程序化复用能力的相关框架（近期应该会有）。

3.3 评估指标

主指标包括任务成功率、决策步数、真实原子调用步数，以及成本或延迟。机制指标包括技能使用比例、技能覆盖率、技能成功率、技能中断比例与中断位置分布，以及不同 select_strategy 的命中率与贡献。

3.4 消融实验

我们进行以下消融：去掉 BPE 预算控制或裁剪；去掉模板化实例化仅保留符号宏；去掉 trace 与软中断；只保留整体奖励不使用阶段反馈；以及技能库规模与技能最大长度的敏感性分析。